

Parte 1 — Paradigmas e conceitos básicos de POO

Questão 1. (assinalar)

Analise as afirmações sobre paradigmas de programação:

- I. No paradigma imperativo, o programa é descrito como uma sequência de comandos que alteram o estado.
- II. O paradigma funcional trata a computação como avaliação de funções e evita estado mutável.
- III. No paradigma orientado a objetos, o código é organizado em objetos, que são instâncias de classes.
- IV. O paradigma lógico baseia-se em fatos e regras (ex.: Prolog).

Assinale a alternativa correta:

- ☐ a) Apenas I e III estão corretas.
- ☐ b) Apenas II e IV estão corretas.
- ☐ c) Apenas I, II e III estão corretas.
- ☒ d) Todas as afirmações estão corretas.
- ☐ e) Nenhuma afirmação está correta.

Questão 2. (soma)

Sobre as vantagens e as contrapartidas do paradigma orientado a objetos, some os números das afirmativas CORRETAS:

- ☒ 01. Favorece a facilidade de manutenção e de evolução do sistema.
- ☐ 02. Impede totalmente a reutilização de código entre projetos.
- ☒ 04. Tende a exigir uma maior curva de aprendizagem em relação a outros paradigmas.
- ☒ 08. Não é recomendável para qualquer tipo de problema.
- ☐ 16. Elimina a necessidade de qualquer planejamento ou modelagem prévia.

Resposta (soma): **13** (01 + 04 + 08)

Questão 3. (assinalar)

Considere o objeto que representa a jogadora "Marta", caracterizado pelos atributos altura = 1,71 e peso = 56. Após uma operação, o peso passa a 56,5. Sobre essa mudança, assinale a alternativa correta:

- ☐ a) O objeto trocou de classe, pois seus dados mudaram.
- ☐ b) Objetos não podem ter seu estado alterado após a criação.
- ☒ c) Houve mudança no estado do objeto, pois o estado corresponde ao valor de seus atributos.
- ☐ d) O valor de um atributo não é considerado um dado.
- ☐ e) A classe passou a ter um novo atributo chamado peso.

Parte 2 — UML e diagrama de classes

Questão 4. (soma)

Sobre a UML e o diagrama de classes, some os números das afirmativas CORRETAS:

- ☒ 01. O diagrama de classes representa uma visão estática do software.
- ☐ 02. Todo sistema deve, obrigatoriamente, ser modelado com os 14 diagramas da UML.
- ☒ 04. A UML é a linguagem padrão de modelagem adotada internacionalmente pela indústria de software.
- ☒ 08. Nomes de classes, atributos e operações não utilizam espaços.



() 16. O diagrama de classes mostra apenas objetos, sem classes.

Resposta (soma): 13 (01 + 04 + 08)

Questão 5. (relacionar colunas)

Relacione cada item ao estilo de nomenclatura correto e assinale a sequência lida de cima para baixo na coluna da direita:

- | | |
|----------------------|---|
| A. SensorTemperatura | (C) Operação em lowerCamelCase, com parênteses. |
| B. saldoAtual | (A) Nome de classe em UpperCamelCase. |
| C. calcularJuros() | (B) Nome de atributo em lowerCamelCase. |

Resposta (sequência de cima para baixo): C - A - B

Questão 6. (assinalar)

Uma operação de uma classe foi modelada em UML como calcularDesconto(). Sobre operações, assinale a alternativa correta:

- () a) O nome deveria ser um substantivo, pois representa uma propriedade.
- () b) Operações não podem retornar valores em hipótese alguma.
- () c) O par de parênteses após o nome é opcional na UML.
- (X) d) Uma operação pode mudar o estado do objeto ou, a partir do estado, produzir um valor de retorno.
- () e) Toda operação deve ser escrita em UpperCamelCase.

Parte 3 — Implementação, memória e diagrama de objetos

Questão 7. (somatória)

Sobre a tradução de classes para Java e os métodos, some os números das afirmativas CORRETAS:

- (X) 01. Atributos são geralmente traduzidos em variáveis de instância.
- (X) 02. Um método que apenas altera o estado do objeto normalmente é declarado como void.
- () 04. Operações da classe nunca podem ser traduzidas em métodos.
- (X) 08. Um método que devolve um valor deve declarar o tipo do dado retornado.
- (X) 16. Um método pode usar o valor dos atributos do próprio objeto em seus cálculos.

Resposta (soma): 27 (01 + 02 + 08 + 16)

Questão 8. (assinalar)

Considere o código:

```
ContaBancaria c1 = new ContaBancaria();  
ContaBancaria c2 = c1;  
c1.depositar(100);
```

Sabendo que a variável de referência armazena o endereço de um objeto, assinale a alternativa correta:

- (X) a) c1 e c2 referenciam o mesmo objeto; o depósito é visível por ambas as variáveis.
- () b) c1 e c2 referenciam objetos diferentes; o depósito afeta apenas c1.
- () c) A linha "ContaBancaria c2 = c1;" cria um novo objeto na Heap.
- () d) O código não compila, pois não é possível atribuir uma referência a outra.
- () e) c2 armazena uma cópia independente dos atributos de c1.

Questão 9. *(somatória)*

Sobre o operador new e as áreas de memória (Stack e Heap), some os números das afirmativas CORRETAS:

- ☒ 01. A Stack armazena variáveis locais, parâmetros de métodos e chamadas de funções.
- ☒ 02. A Heap armazena os objetos.
- ☐ 04. O operador new é usado para declarar variáveis de tipo primitivo, como int.
- ☒ 08. O operador new aloca espaço para as variáveis de instância do objeto.
- ☒ 16. O operador new retorna o endereço de memória do objeto criado.

Resposta (soma): 27 (01 + 02 + 08 + 16)

Questão 10. *(assinalar)*

Sobre o diagrama de objetos, assinale a alternativa correta:

- ☐ a) Contém somente classes e seus relacionamentos.
- ☐ b) É usado para modelar o fluxo de exceções do programa.
- ☐ c) Não pode conter valores de atributos.
- ☐ d) É idêntico ao diagrama de classes, apenas com outro nome.
- ☒ e) Mostra uma fotografia do estado do sistema em um instante, contendo somente objetos.

Parte 4 — Escopo de variáveis

Questão 11. *(assinalar)*

Considere:

```
public class Teste {  
    int x;  
    void m() {  
        int y = 5;  
        {  
            int z = 2;  
        }  
        System.out.println(z);  
    }  
}
```

Assinale a alternativa correta:

- ☐ a) O código compila e imprime 2.
- ☐ b) O código imprime 0, pois z é variável de instância inicializada com zero.
- ☒ c) Erro de compilação: z está fora de escopo na linha do println (existe apenas dentro do bloco).
- ☐ d) Erro de compilação porque x não foi utilizado.
- ☐ e) O código imprime 5.

Questão 12. *(somatória)*

Sobre inicialização e valores padrão de variáveis, some os números das afirmativas CORRETAS:

- ☒ 01. Uma variável local não possui valor inicial e só pode ser lida após atribuição explícita.
- ☒ 02. Uma variável de instância do tipo int é inicializada automaticamente com 0.
- ☐ 04. Toda variável de bloco recebe automaticamente o valor 0 ao ser declarada.
- ☒ 08. Uma variável de instância do tipo boolean é inicializada automaticamente com false.
- ☒ 16. Uma variável de instância de referência é inicializada automaticamente com null.

Resposta (soma): 27 (01 + 02 + 08 + 16)

Questão 13. *(assinalar)*

Considere:

```
public class C {  
    int n = 100;  
    void mostrar() {  
        int n = 7;  
        System.out.println(n);  
        System.out.println(this.n);  
    }  
}
```

O que é impresso ao chamar mostrar()?

- ☐ a) 100 e depois 7.
- ☒ b) 7 e depois 100.
- ☐ c) 7 e depois 7.
- ☐ d) 100 e depois 100.
- ☐ e) Erro de compilação, pois há duas variáveis n.

Parte 5 — Encapsulamento

Questão 14. *(soma)*

Sobre encapsulamento, some os números das afirmativas CORRETAS:

- ☒ 01. O encapsulamento visa controlar o acesso aos atributos para garantir a integridade dos dados.
- ☒ 02. Em Java, encapsular um atributo significa, tipicamente, declará-lo como private.
- ☒ 04. Recomenda-se encapsular todos os atributos; public só é admissível para constantes.
- ☐ 08. Tornar um atributo público é a forma recomendada de garantir seu encapsulamento.
- ☒ 16. Atributos privados podem ser expostos a outras classes por meio de getters.

Resposta (soma): 23 (01 + 02 + 04 + 16)

Questão 15. *(assinalar)*

Considere a intenção de expor, para leitura, um atributo privado boolean ativo. Assinale a assinatura de método de acesso mais adequada segundo a convenção vista em aula:

- ☐ a) public void getAtivo(boolean ativo)
- ☐ b) private boolean ativo()
- ☒ c) public boolean isAtivo()
- ☐ d) public void setAtivo()
- ☐ e) public boolean getAtivo(boolean valor)

Questão 16. *(assinalar)*

Considere:

```
private double saldo;  
public void setSaldo(double saldo) {  
    saldo = saldo;  
}
```

O setter acima não altera o atributo da classe. Qual a correção adequada?

- ☐ a) Declarar o atributo saldo como static.
- ☐ b) Trocar o tipo do parâmetro para int.



() c) Renomear o método para getSaldo.

(X) d) Usar this.saldo = saldo; para distinguir o atributo do parâmetro de mesmo nome.

() e) Remover o modificador private do atributo.

Questão 17. *(relacionar colunas)*

Relacione o símbolo de visibilidade da UML ao seu significado e assinale a sequência da coluna da direita (de cima para baixo):

- | | |
|------------------|--|
| A. - (menos) | (C) protected — protegido. |
| B. + (mais) | (A) private — somente a própria classe. |
| C. # (sustenido) | (B) public — qualquer classe. |

Resposta (sequência de cima para baixo): C - A - B

Parte 6 — Membros de classe, sobrecarga e construtores

Questão 18. *(soma)*

Sobre membros de classe (static), some os números das afirmativas CORRETAS:

(X) 01. Uma variável static é compartilhada entre todas as instâncias da classe.

() 02. Variáveis static são exclusivas de cada instância, não compartilhadas.

(X) 04. Membros de classe podem ser usados sem que exista uma instância da classe.

(X) 08. Um método static não pode utilizar a palavra this.

(X) 16. Um método static não pode manipular variáveis de instância sem uma instância explícita.

Resposta (soma): 29 (01 + 04 + 08 + 16)

Questão 19. *(assinalar)*

Considere as tentativas de sobrecarga na mesma classe:

```
int somar(int a, int b) { ... }  
double somar(int a, int b) { ... }
```

Assinale a alternativa correta:

() a) É uma sobrecarga válida, pois os tipos de retorno são diferentes.

() b) É válida porque os nomes são iguais.

(X) c) Não é válida: o compilador não diferencia métodos apenas pelo tipo de retorno; as assinaturas são iguais.

() d) É válida apenas se um dos métodos for static.

() e) É válida porque a sobrecarga ignora a lista de parâmetros.

Questão 20. *(soma)*

Sobre construtores em Java, some os números das afirmativas CORRETAS:

(X) 01. Um construtor tem o mesmo nome (identificador) da classe.

(X) 02. Um construtor não declara tipo de dado de retorno.

() 04. Todo construtor deve declarar void como tipo de retorno.

(X) 08. Se nenhum construtor for escrito, o compilador fornece um construtor padrão sem argumentos.

(X) 16. O construtor é executado pelo operador new durante a criação do objeto.

Resposta (soma): 27 (01 + 02 + 08 + 16)

Parte 7 — Lançamento e tratamento de exceções

Questão 21. (assinalar)

Considere:

```
try {  
    int[] v = new int[2];  
    v[4] = 30 / Integer.parseInt("0");  
} catch (ArithmeticException e) {  
    System.out.println("A");  
} catch (ArrayIndexOutOfBoundsException e) {  
    System.out.println("B");  
}
```

Qual é a saída? (Dica: a divisão é avaliada antes da atribuição ao vetor.)

☒ a) A

☐ b) B

☐ c) A e depois B

☐ d) Nenhuma saída; o programa encerra normalmente.

☐ e) Erro de compilação.

Questão 22. (somatória)

Sobre o comando try..catch..finally e a hierarquia de exceções, some os números das afirmativas CORRETAS:

☒ 01. Se nenhum erro ocorrer no bloco try, os comandos do catch não são executados.

☒ 02. A cláusula finally é executada quer ocorra erro ou não no try.

☐ 04. A cláusula finally só é executada quando uma exceção é lançada.

☒ 08. A classe Exception e suas subclasses indicam erros que poderiam ser tratados pelo programa.

☒ 16. Uma exceção pode ser tratada por um catch cujo parâmetro seja uma superclasse do erro (polimorfismo).

Resposta (soma): **27** (01 + 02 + 08 + 16)

Questão 23. (assinalar)

Considere a ordem das cláusulas catch:

```
try {  
    // ...  
} catch (Exception e) {  
    // ...  
} catch (NumberFormatException e) {  
    // ...  
}
```

Assinale a alternativa correta:

☐ a) Compila normalmente e cada catch trata seu erro específico.

☐ b) Erro: a classe Exception não pode ser usada em catch.

☐ c) O segundo catch é executado sempre antes do primeiro.

☒ d) Erro: não é válido declarar um catch de subclasse (NumberFormatException) após um catch de sua superclasse (Exception).

☐ e) É obrigatório existir um finally nesse caso.



Questão 24. *(relacionar colunas)*

Relacione o mecanismo de exceção à sua descrição e assinale a sequência da coluna da direita (de cima para baixo):

- | | |
|---------------------------------|---|
| A. throw | (B) Delega ao método chamador o tratamento de exceções verificáveis. |
| B. throws | (D) Deve ser tratada ou postergada; não é subclasse de RuntimeException nem de Error. |
| C. finally | (A) Lança explicitamente um objeto de exceção. |
| D. Exceção verificada (checked) | (C) Bloco sempre executado, tipicamente para liberar recursos. |

Resposta (sequência de cima para baixo): B - D - A - C

Parte 8 — Qualidade de software e testes de unidade

Questão 25. *(soma)*

Sobre qualidade de software, testes de unidade e JUnit, some os números das afirmativas CORRETAS:

- (X) 01. Quanto mais cedo um erro é detectado no ciclo de desenvolvimento, menor o custo de correção.
- (X) 02. Em POO, a unidade testada em um teste de unidade é o método.
- (X) 04. No JUnit, o método de teste usa a anotação @Test, é público, sem parâmetros e retorna void.
- () 08. assertEquals sempre compara o sistema inteiro em ambiente de produção.
- (X) 16. assertThrows é usado para verificar se uma exceção esperada foi lançada.

Resposta (soma): 23 (01 + 02 + 04 + 16)